1. Linear Models and Linear Algebra

This lecture introduces foundational concepts in machine learning and linear algebra that will recur throughout the course. We begin by distinguishing between **supervised and unsupervised learning** paradigms, and focus on the first. Within supervised learning, we introduce the formal distinction between **regression and classification** problems, which differ in the structure of their output spaces. We then discuss **linear and logistic regression** as core supervised learning models and show how they can be expressed compactly using vector and matrix operations. To support this, we introduce the **essential linear algebra** required to represent data, model parameters, and predictions efficiently. These mathematical tools form the basis for all subsequent models in the course, including neural network architectures.

Learning Paradigms

In machine learning, different learning paradigms are defined by the type of *supervision* available during training. Supervision refers to the presence of target variables associated with training inputs, which provide an explicit learning signal indicating the desired output for each input.

For example, consider a word-sense disambiguation task. In a supervised setting, each occurrence of a polysemous word is paired with a *label* indicating its intended sense in context:

(1a) The data are stored in the *cloud*.
(Computing Infrastructure)

(1b) Dark clouds gathered over the city.
(Meteorological Phenomenon)

These sense labels provide supervision in the sense that they specify the linguistically appropriate interpretation for each training example, *i.e.*, each occurrence of the word. In an unsupervised setting, the learner would be given only raw text containing occurrences of the ambiguous word, without sense annotations, and it would typically

be tasked with discovering clusters corresponding to different usages.

More formally, in *supervised learning* the learner is provided with a dataset of input–output pairs

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}, \tag{1}$$

where each input vector[1,2]

$$\mathbf{x}^{(i)} = \left(x_1^{(i)}, \ldots, x_d^{(i)}\right) \in \mathbb{R}^d \tag{2}$$

is associated with a target value $y^{(i)} \in \mathcal{Y}$. The set $\mathcal{Y}$ denotes the *output space* (or *label space*) of the learning problem, that is, the set of all values that the target variable may take. Its structure depends on the task, as we will see in the next section on regression and classification.

The goal of supervised learning is to learn a function

$$f : \mathbb{R}^d \to \mathcal{Y} \tag{3}$$

that generalises beyond the training data and produces accurate predictions on previously unseen inputs.

In *unsupervised learning*, by contrast, the learner is given only input data

$$\{\mathbf{x}^{(i)}\}_{i=1}^{N}, \tag{4}$$

without corresponding target labels. The goal is not to predict predefined outputs, but rather to discover structure in the data, such as clusters, latent variables, or low-dimensional representations.

This course focuses primarily on supervised learning, as it underlies most core computational linguistics and NLP tasks, including classification, tagging, structured prediction, and language modelling.

Regression and Classification

Within supervised learning, learning problems are typically categorised as either *regression* or *classification*, according to the type of values the model is required to predict.

In *regression*, the target variable is continuous. Formally, the output space satisfies

$$\mathcal{Y} \subseteq \mathbb{R}, \tag{5}$$

and the goal is to learn a function

$$f : \mathbb{R}^d \to \mathbb{R}. \tag{6}$$

[1] We will introduce this notation more in detail below. For now, $\mathbb{R}$ denotes the set of real numbers (for example -1, 0, or 3.14). The notation $\mathbb{R}^d$ refers to the set of all ordered collections (vectors) of d real numbers. Thus, an element $\mathbf{x} \in \mathbb{R}^d$ can be thought of simply as a list of d numerical values, where the order of the values matters.

[2] Throughout this chapter, superscripts (i) are used to index *data points*, while subscripts index *features*.

Regression models therefore predict real-valued quantities. In computational linguistics, regression problems occur in tasks such as predicting human judgements of sentence acceptability or word usage similarity, or estimating reading or reaction times. More generally, regression is used whenever linguistic phenomena are modelled as scalar quantities rather than categorical labels.

In *classification*, by contrast, the target variable is discrete and drawn from a finite set of labels,

$$\mathcal{Y} = \{1, \ldots, K\}. \tag{7}$$

The goal is to learn a function

$$f : \mathbb{R}^d \to \{1, \ldots, K\} \tag{8}$$

that assigns each input to exactly one class.

Many core NLP tasks are naturally framed as classification problems. Examples include sentiment analysis, topic classification, intent recognition, part-of-speech tagging, and word-sense disambiguation. Next-word prediction in language modelling can also be viewed as a classification problem: given a linguistic context, the model selects one item from a finite vocabulary as the next word. In this case, the set of classes corresponds to the vocabulary of the language.

Linear Models for Regression and Classification

In this section, we introduce a class of supervised learning models in which predictions are based on linear functions of the input representation. These models form the conceptual and mathematical foundation for much of modern machine learning, and they provide a useful starting point for understanding more complex architectures.

We begin with linear regression, which is used to predict continuous target values, and then show how the same linear framework can be extended to classification through logistic and multinomial logistic regression. Although these models differ in the structure of their output spaces and loss functions, they share a common underlying form: a linear combination of input features.

Linear Regression

Linear regression is one of the simplest and most widely used supervised learning models. Its purpose is to predict a real-valued output from a fixed set of numerical input features.

Formally, each input is represented as an ordered collection of d real-valued quantities,

$$\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_d^{(i)}), \tag{9}$$

where each component $x_j^{(i)}$ corresponds to a particular measurable property of the input. The model produces a prediction $\widehat{y}^{(i)}$ by computing a linear function of these inputs,

$$\widehat{y}^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots + w_d x_d^{(i)} \tag{10}$$

$$= w_0 + \sum_{j=1}^{d} w_j \, x_j^{(i)}. \tag{11}$$

The parameters $w_1, \ldots, w_d$ are referred to as *weights* and determine the contribution of each input feature to the prediction. The parameter w_0 is called the *intercept* and represents the predicted output when all input features take the value zero. The output $\widehat{y}^{(i)} \in \mathbb{R}$ is the model's prediction and is intended to approximate an observed target value $y^{(i)}$.

Training a linear regression model consists in selecting the parameter values $w_0, \ldots, w_d$ so as to minimise the discrepancy between the predicted values $\{\widehat{y}^{(i)}\}_{i=1}^{N}$ and the true target values $\{y^{(i)}\}_{i=1}^{N}$ observed in the data.

A standard way to quantify this discrepancy is the *mean squared error* (MSE). MSE measures the average squared difference between predicted and true values and is defined as:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (\widehat{y}^{(i)} - y^{(i)})^2, \tag{12}$$

where N denotes the number of test examples. The squaring operation penalises larger errors more strongly. Lower MSE values indicate better predictive performance.

Example: Predicting Reading Time. Consider the task of predicting the reading time[3] of a word in a sentence, measured in milliseconds. Suppose that for each word we extract three numerical features: its length (measured in number of characters), its frequency in a corpus, and its contextual predictability, quantified as surprisal (negative log probability). For a given word i, the input representation may then take the form

$$\mathbf{x}^{(i)} = (\text{length}^{(i)}, \text{frequency}^{(i)}, \text{surprisal}^{(i)}) = (5,\ 1200,\ 6.3), \tag{13}$$

where word i has five characters, occurs 1200 times in the corpus, and has surprisal value 6.3. The corresponding output is a real-valued reading time—for example,

$$y^{(i)} = 245, \tag{14}$$

indicating that word i was read in 245 milliseconds. The goal of the linear regression model is to use such numerical input features,

[3] In psycholinguistic research, there are several standard measures of reading time based on eye-tracking data. Let us assume reading time is operationalised as *total fixation duration*, that is, the sum of the durations of all eye fixations on a word during sentence reading.

across multiple words in a dataset, to predict reading times as accurately as possible. In practice, this is achieved by adjusting the weights so that the model's predicted reading times are as close as possible to the empirically measured reading times—*i.e.*, such that the mean squared error is minimised.

Logistic Regression

Logistic regression extends the linear regression framework to *binary classification* problems, in which the target variable can take one of two possible values. Rather than predicting a continuous quantity, the goal is to predict which of two classes an input belongs to.

As in linear regression, the model begins by computing a linear function of the input features. For an input representation

$$\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_d^{(i)}), \tag{15}$$

the model computes a linear score

$$z^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots + w_d x_d^{(i)} \tag{16}$$

$$= w_0 + \sum_{j=1}^{d} w_j \, x_j^{(i)}. \tag{17}$$

This score is not itself a probability and can take any real value. To convert it into a quantity that can be interpreted probabilistically, logistic regression applies a fixed non-linear transformation known as the *logistic* (or *sigmoid*) function,

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{18}$$

The logistic function maps any real-valued input to a value strictly between 0 and 1. For large positive values of z, $\sigma(z)$ approaches 1, while for large negative values of z, $\sigma(z)$ approaches 0. This makes it suitable for modelling probabilities.

The output $\sigma(z^{(i)})$ is interpreted as the probability that the input belongs to the positive class. A binary prediction is then obtained by comparing this probability to a fixed threshold, typically 0.5:

$$\widehat{y}^{(i)} = \begin{cases} 1 & \text{if } \sigma(z^{(i)}) > 0.5, \\ 0 & \text{otherwise.} \end{cases} \tag{19}$$

Classification models are commonly evaluated using *accuracy*, which measures the proportion of correctly predicted labels. Formally, given a dataset of N examples with true labels $y^{(i)}$ and predicted labels $\widehat{y}^{(i)}$, accuracy is defined as

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\widehat{y}^{(i)} = y^{(i)}], \tag{20}$$

where $\mathbb{1}[\cdot]$ is the *indicator function*, which equals 1 if its argument is true and 0 otherwise. Accuracy therefore takes values between 0 and 1, with higher values indicating better classification performance.

Logistic (Sigmoid) Function. The logistic function, often called the sigmoid function, is a common activation function used to map any real-valued number into the range $(0, 1)$. It is defined by the formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{21}$$

It is well-suited for predicting probabilities and is the core activation function for binary classification. Its characteristic "S-shaped" curve is shown in Figure 1.

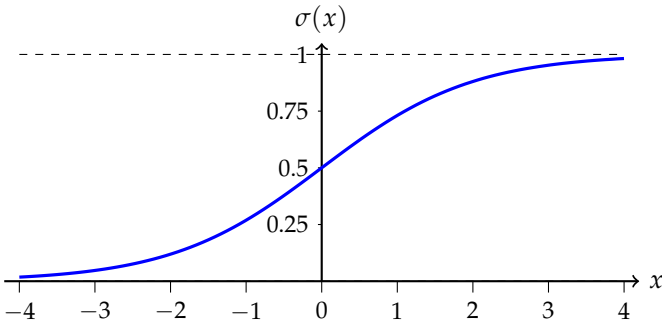

Figure 1: The logistic function squashes input values between 0 and 1. It is symmetric around the point $(0, 0.5)$.

Example: Predicting Word Skipping. Consider the task of predicting whether a word is skipped during reading. In eye-tracking studies, a word is said to be *skipped* if it is not fixated at all during a reader's first pass through the sentence. This yields a binary outcome: either the word is skipped or it is fixated.

Using the same predictors as before, the input representation for a given word i may be

$$\mathbf{x}^{(i)} = (\text{length}^{(i)}, \text{frequency}^{(i)}, \text{surprisal}^{(i)}), \tag{22}$$

and the target variable is

$$y^{(i)} \in \{0, 1\}, \tag{23}$$

where $y^{(i)} = 1$ indicates that the word was skipped and $y^{(i)} = 0$ indicates that it was fixated. Logistic regression models the probability that a word will be skipped as a function of its length, frequency, and contextual predictability.

Multinomial Logistic Regression

Many classification problems involve more than two possible output labels. In such cases, logistic regression can be generalised to *multinomial logistic regression*, which models a discrete choice among a finite set of classes.

As before, each input is represented as an ordered collection of features,

$$\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_d^{(i)}). \tag{24}$$

Rather than computing a single score, the model now computes a separate linear score for each possible class. For class $k \in \{1, \ldots, K\}$, this score is given by

$$z_k^{(i)} = w_{0,k} + w_{1,k}x_1^{(i)} + w_{2,k}x_2^{(i)} + \cdots + w_{d,k}x_d^{(i)} \tag{25}$$

$$= w_{0,k} + \sum_{j=1}^{d} w_{j,k}\, x_j^{(i)}, \tag{26}$$

where each class has its own set of parameters. As in logistic regression, these scores can take any real value and are not directly interpretable as probabilities.

To convert the set of scores $\{z_1^{(i)}, \ldots, z_K^{(i)}\}$ into a probability distribution over classes, multinomial logistic regression applies the *softmax* function,

$$P(y^{(i)} = k \mid \mathbf{x}^{(i)}) = \frac{\exp(z_k^{(i)})}{\sum_{j=1}^{K} \exp(z_j^{(i)})}. \tag{27}$$

The softmax function maps a vector of real-valued scores to a vector of non-negative values that sum to 1. Each output can therefore be interpreted as the probability that the input belongs to class k. The predicted class is typically taken to be the one with the highest probability,[4] according to the decision rule

$$\widehat{y}^{(i)} = \operatorname*{argmax}_{k \in \{1,\ldots,K\}} P(y^{(i)} = k \mid \mathbf{x}^{(i)}). \tag{28}$$

Softmax Function. The softmax function generalises the logistic function to multiple classes. It converts a vector of K real numbers (logits) into a probability distribution of K possible outcomes. Each element of the output vector is defined as:

$$\text{softmax}(\mathbf{z})_k = \frac{\exp(z_k^{(i)})}{\sum_{j=1}^{K} \exp(z_j^{(i)})}. \tag{29}$$

The outputs are non-negative and sum to 1, making them interpretable as probabilities. Figure 2 illustrates the effect of softmax on a sample 3-class logit vector $\mathbf{z} = [2.0, 1.0, 0.1]^\top$.

[4] An alternative approach to multi-class classification is to train multiple binary logistic regression models, one for each class, rather than a single multinomial model. In a *one-vs-rest* (or *one-vs-all*) setup, each classifier predicts whether the input belongs to a given class or not, and the final prediction is obtained by selecting the class with the highest predicted probability. In a *one-vs-one* setup, a separate classifier is trained for each pair of classes, and predictions are combined by majority voting. While these approaches can be effective, they do not yield a single, jointly normalised probability distribution over classes, unlike multinomial logistic regression with a softmax output.

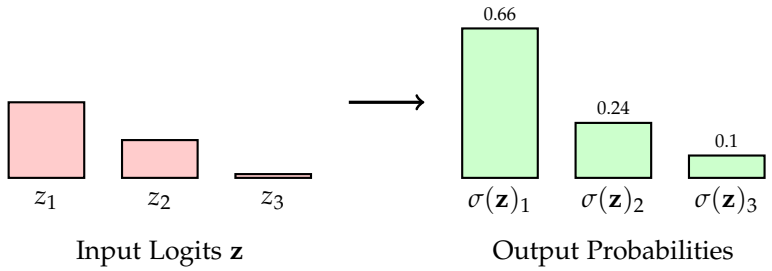

Figure 2: The softmax function normalizes input logits into a probability distribution. The largest logit corresponds to the highest probability, but all outputs are positive and sum to 1.

Example: Part-of-Speech Tagging. As a linguistic example, consider part-of-speech tagging. In this task, the goal is to assign each word in a sentence a syntactic category, such as NOUN, VERB, or ADJECTIVE, from a finite tag set. For a given word i, the relevant information is typically a collection of *categorical* properties of the local context and word form, such as the part-of-speech tag of the previous word, the suffix of the current word, and the prefix of the current word. These properties are not numerical by themselves and therefore cannot be used directly by a linear model.

Instead, each possible feature–value pair is represented as a separate binary indicator in a high-dimensional vector. Concretely, we construct an input representation

$$\mathbf{x}^{(i)} \in \{0,1\}^d, \tag{30}$$

where each dimension corresponds to a specific condition such as "previous tag = NOUN", "suffix = `-ing`", or "prefix = `un-`". The vector $\mathbf{x}^{(i)}$ contains a 1 for the small number of feature–value pairs that are active for word i, and 0 everywhere else. Because only a tiny fraction of the d dimensions are non-zero for any given word, this representation is called *sparse*. Sparse representations allow linear and multinomial logistic regression models to operate on rich categorical feature spaces.

The output space is a finite *tag set* $\mathcal{Y}$, where each class corresponds to one possible part-of-speech tag. Multinomial logistic regression models the probability of each possible tag given the sparse input vector and predicts the tag with the highest probability.

Linear Algebra

As learning models become more complex and datasets larger, writing expressions as sums over individual features quickly becomes impractical. Linear algebra provides a compact, systematic notation for expressing regression and classification models, and it closely reflects how these models are implemented computationally.

More importantly, linear algebra allows us to reason about entire datasets, parameter collections, and predictions at once, rather than one feature or one data point at a time. For this reason, essentially all modern machine learning models are formulated in linear-algebraic terms.

In this section, we review the core linear algebra concepts required to express linear models cleanly and efficiently.

Vectors, Matrices, and Their Dimensions

A *vector* is an ordered collection of real numbers. An n-dimensional vector is written as

$$\mathbf{a} \in \mathbb{R}^n. \tag{31}$$

By convention, we treat vectors as column vectors,

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}. \tag{32}$$

In machine learning, vectors are used to represent input examples, parameter values, and intermediate quantities such as neural network activations. For example, a feature representation of a word might be encoded as a vector whose components correspond to measurable linguistic properties.

A *matrix* is a two-dimensional array of real numbers. A matrix with n rows and m columns is written as

$$A \in \mathbb{R}^{n \times m}, \tag{33}$$

and can be written explicitly as

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,m} \\ a_{2,1} & a_{2,2} & \dots & a_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \dots & a_{n,m} \end{pmatrix}. \tag{34}$$

Matrices are commonly used to represent collections of vectors, linear transformations, or model parameters. For instance, a dataset consisting of N input vectors of dimensionality d can be represented as a data matrix

$$A \in \mathbb{R}^{N \times d}, \tag{35}$$

where each row corresponds to one input example and each column corresponds to a feature.

Dot Product

One of the most fundamental operations between two vectors of equal length is the *dot product*. Given two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, their dot product is defined as

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^{n} a_i b_i. \tag{36}$$

The dot product produces a scalar value and measures the degree to which two vectors align. In linear models, it plays a central role, as predictions are obtained by computing the dot product between a feature vector and a weight vector.

For example, if

$$\mathbf{w} = \begin{pmatrix} 10 \\ -0.1 \\ 3.5 \end{pmatrix}, \qquad \mathbf{x} = \begin{pmatrix} 5 \\ 1200 \\ 6.3 \end{pmatrix}, \tag{37}$$

then the dot product is

$$\begin{aligned} \mathbf{w}^\top \mathbf{x} &= w_1 x_1 + w_2 x_2 + w_3 x_3 && (38)\\ &= 10 \cdot 5 - 0.1 \cdot 1200 + 3.5 \cdot 6.3 && (39)\\ &= 50 - 120 + 22.05 && (40)\\ &= -48.05. && (41) \end{aligned}$$

Matrix Transposition

The *transpose* of a matrix $A \in \mathbb{R}^{n \times m}$, denoted $A^\top$, is obtained by swapping rows and columns. The resulting matrix has shape $m \times n$ and is defined elementwise as

$$(A^\top)_{i,j} = A_{j,i}. \tag{42}$$

Transposition is frequently used to ensure that dimensions match correctly in matrix expressions. In particular, it allows us to switch between row-vector and column-vector representations of the same underlying data.

For example, consider the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2 \times 3}. \tag{43}$$

Its transpose is

$$A^\top = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix} \in \mathbb{R}^{3 \times 2}. \tag{44}$$

Matrix Multiplication

Matrix multiplication generalises the dot product to matrices. Let

$$A \in \mathbb{R}^{m \times n}, \tag{45}$$

$$B \in \mathbb{R}^{n \times p}. \tag{46}$$

Their product

$$C = AB \in \mathbb{R}^{m \times p} \tag{47}$$

is defined elementwise by

$$C_{i,j} = A_{i,1}B_{1,j} + A_{i,2}B_{2,j} + \cdots + A_{i,n}B_{n,j} \tag{48}$$

$$= \sum_{k=1}^{n} A_{i,k}B_{k,j}. \tag{49}$$

Matrix multiplication is only defined when the number of columns of A matches the number of rows of B. This ensures that each entry of the product is formed by a valid dot product.

For example, let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \in \mathbb{R}^{2\times 2}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \in \mathbb{R}^{2\times 2}. \tag{50}$$

Then

$$AB = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix} \tag{51}$$

$$= \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}. \tag{52}$$

Matrix–vector multiplication

A particularly important special case of matrix multiplication occurs when the right-hand factor is a vector $\mathbf{b} \in \mathbb{R}^n$. In this case,

$$\mathbf{a} = A\mathbf{b} \in \mathbb{R}^m. \tag{53}$$

Written out explicitly,

$$\mathbf{a} = \begin{pmatrix} A_{1,1} & A_{1,2} & \dots & A_{1,n} \\ A_{2,1} & A_{2,2} & \dots & A_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m,1} & A_{m,2} & \dots & A_{m,n} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}. \tag{54}$$

The ith component of $\mathbf{a}$ is

$$a_i = A_{i,1}b_1 + A_{i,2}b_2 + \cdots + A_{i,n}b_n \tag{55}$$

$$= \sum_{k=1}^{n} A_{i,k}b_k. \tag{56}$$

Thus, each output component is the dot product between the ith row of A and the vector $\mathbf{b}$. Equivalently, $\mathbf{a}$ can be interpreted as a linear combination of the columns of A, with coefficients given by the components of $\mathbf{b}$. Written out explicitly, this column-wise view is

$$\mathbf{a} = b_1 \begin{pmatrix} A_{1,1} \\ A_{2,1} \\ \vdots \\ A_{m,1} \end{pmatrix} + b_2 \begin{pmatrix} A_{1,2} \\ A_{2,2} \\ \vdots \\ A_{m,2} \end{pmatrix} + \cdots + b_n \begin{pmatrix} A_{1,n} \\ A_{2,n} \\ \vdots \\ A_{m,n} \end{pmatrix} \tag{57}$$

$$= \sum_{k=1}^{n} b_k \begin{pmatrix} A_{1,k} \\ A_{2,k} \\ \vdots \\ A_{m,k} \end{pmatrix}. \tag{58}$$

Linear Models in Matrix Form

So far, we have introduced linear and logistic regression using explicit sums over features. While this notation is useful for understanding the role of individual parameters, it quickly becomes unwieldy when dealing with high-dimensional input representations, large datasets, or multiple output classes. Linear algebra provides a more compact and expressive way to formulate these models, which also closely mirrors their computational implementation.

This section reformulates linear regression, logistic regression, and multinomial logistic regression using vectors and matrices.

Linear Regression in Vector Form

Recall the linear regression model for a single input example with d features,

$$\widehat{y}^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots + w_d x_d^{(i)}. \tag{59}$$

This expression can be written more compactly by introducing an augmented feature vector and a weight vector. Let

$$\mathbf{x}^{(i)} = \begin{pmatrix} 1 \\ x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{pmatrix} \in \mathbb{R}^{d+1}, \qquad \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{pmatrix} \in \mathbb{R}^{d+1}. \tag{60}$$

The leading constant 1 in the input vector accounts for the intercept term w_0.

Using this notation, the prediction can be written as a dot product between the augmented input vector and the weight vector

$$\widehat{y}^{(i)} = \mathbf{x}^{(i)} \cdot \mathbf{w} = (\mathbf{x}^{(i)})^\top \mathbf{w} = \mathbf{w}^\top \mathbf{x}^{(i)} \in \mathbb{R} \tag{61}$$

The most common way this dot product is written out is as the matrix multiplication $\mathbf{w}^\top \mathbf{x}^{(i)}$, where the weight vector is transposed into a row vector and multiplied by the input column vector.

Linear Regression for Multiple Examples

In practice, we usually work with many input examples at once. Suppose we have a dataset of N examples, each represented by d features. We collect all augmented input vectors into a *data matrix* (or *feature matrix*) consisting of the augmented input vectors:

$$X = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(N)})^\top \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & \dots & x_d^{(N)} \end{pmatrix} \in \mathbb{R}^{N \times (d+1)}. \tag{62}$$

Predictions for all examples can then be computed simultaneously as

$$\widehat{\mathbf{y}} = X\mathbf{w}, \tag{63}$$

where $\widehat{\mathbf{y}} \in \mathbb{R}^N$ is the vector of predicted outputs. Each component of $\widehat{\mathbf{y}}$ corresponds to the prediction for one input example.

Logistic Regression in Matrix Form

Logistic regression uses the same linear structure as linear regression but applies a non-linear transformation to obtain probabilities for binary classification. For a single input example, the linear score is

$$z^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} \in \mathbb{R}. \tag{64}$$

This score is then passed through the logistic function σ, yielding

$$\widehat{p}^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}) \in (0, 1), \tag{65}$$

which is interpreted as the probability of the positive class.

For a dataset of N examples collected in a matrix X, the vector of scores is

$$\mathbf{z} = X\mathbf{w} \in \mathbb{R}^N \tag{66}$$

and probabilities are obtained by applying the sigmoid function elementwise,

$$\widehat{\mathbf{p}} = \sigma(\mathbf{z}) \in \mathbb{R}^N, \tag{67}$$

that is,

$$\widehat{\mathbf{p}} = \begin{pmatrix} \sigma\left(z^{(1)}\right) \\ \sigma\left(z^{(2)}\right) \\ \vdots \\ \sigma\left(z^{(N)}\right) \end{pmatrix} = \begin{pmatrix} \frac{1}{1+\exp\left(-z^{(1)}\right)} \\ \frac{1}{1+\exp\left(-z^{(2)}\right)} \\ \vdots \\ \frac{1}{1+\exp\left(-z^{(N)}\right)} \end{pmatrix}. \tag{68}$$

A binary prediction is then obtained by thresholding each probability $\widehat{p}^{(i)}$, typically at 0.5.

Multinomial Logistic Regression in Matrix Form

Multinomial logistic regression extends the binary case by learning a separate set of weights for each class. Suppose there are K classes. For each class $k \in \{1, \ldots, K\}$, we associate a weight vector

$$\mathbf{w}^{(k)} = \begin{pmatrix} w_0^{(k)} \\ w_1^{(k)} \\ \vdots \\ w_d^{(k)} \end{pmatrix} \in \mathbb{R}^{d+1}, \tag{69}$$

where subscripts index feature dimensions and the superscript (k) indexes classes (*not* individual data points). We collect these class-specific weight vectors into a single weight matrix $W \in \mathbb{R}^{(d+1)\times K}$:

$$W = \begin{pmatrix} \mathbf{w}^{(1)} & \mathbf{w}^{(2)} & \ldots & \mathbf{w}^{(K)} \end{pmatrix} = \begin{pmatrix} w_0^{(1)} & w_0^{(2)} & \ldots & w_0^{(K)} \\ w_1^{(1)} & w_1^{(2)} & \ldots & w_1^{(K)} \\ \vdots & \vdots & \ddots & \vdots \\ w_d^{(1)} & w_d^{(2)} & \ldots & w_d^{(K)} \end{pmatrix} \tag{70}$$

Single example. For a single input example $\mathbf{x}^{(i)} \in \mathbb{R}^{d+1}$, the vector of class scores is obtained by

$$\mathbf{z}^{(i)} = W^\top \mathbf{x}^{(i)} \in \mathbb{R}^K, \tag{71}$$

where the kth component is

$$z_k^{(i)} = \left(\mathbf{w}^{(k)}\right)^\top \mathbf{x}^{(i)} \tag{72}$$

$$= w_0^{(k)} + \sum_{j=1}^{d} w_j^{(k)} x_j^{(i)}. \tag{73}$$

These scores are converted into a probability distribution over classes using the softmax function,

$$P\left(y^{(i)} = k \mid \mathbf{x}^{(i)}\right) = \frac{\exp\left(z_k^{(i)}\right)}{\sum_{j=1}^{K} \exp\left(z_j^{(i)}\right)}. \tag{74}$$

Multiple examples. For a dataset of N examples, we stack the augmented input vectors row-wise into the data matrix

$$X = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(N)})^\top \end{pmatrix} \in \mathbb{R}^{N \times (d+1)}. \tag{75}$$

Scores for all examples and all classes are then computed simultaneously as

$$Z = XW \in \mathbb{R}^{N \times K}, \tag{76}$$

where the entry $Z_{i,k}$ corresponds to the score $z_k^{(i)}$.

Applying the softmax function row-wise to Z yields a matrix of class probabilities, in which each row corresponds to one input example and forms a valid probability distribution over the K classes.